MCPulse --- MCP Server Monitoring & Performance Analytics Platform

Purpose: Project specification and implementation guide for Antigravity\ **Domain:** Cloud Computing\ **Deadline:** Complete within the current semester\ **Approach:** Cloud-based monitoring of remotely accessible MCP servers

1. Project summary

MCPulse is a cloud-hosted monitoring and analytics platform for remotely accessible Model Context Protocol (MCP) servers. It periodically checks configured MCP server endpoints, records availability and response-time measurements, captures failures, and presents current status, historical trends, and incidents in a web dashboard.

The primary academic focus is **Cloud Computing**: deploying and operating an application and monitoring workload on cloud infrastructure; using cloud networking, identity/access control, data storage, logs, and metrics where available. MCP is the monitored workload and motivating use case.

2. Problem statement

Operators of remotely accessible MCP servers may lack a single place to view service status, response times, and historical failures. MCPulse aims to provide that visibility through scheduled checks, stored measurements, incident records, and a dashboard.

3. Goals

- Provide a dashboard listing configured MCP servers and their latest status.
- Run scheduled checks against remotely reachable MCP endpoints using the selected server's supported transport/protocol.
- Record check time, outcome, duration, and sanitized error details.
- Show historical response-time and availability trends.
- Create and resolve incident records based on documented failure/recovery rules.
- Deploy to AWS where the Learner Lab permits.
- Demonstrate cloud compute, networking, access control, persistence, and operational monitoring.

4. Scope and exclusions

Included in the initial version

- MCP server registry: add, view, edit, enable/disable, and delete server configurations.
- Scheduled health/protocol checks.
- Response-time measurement and check history.
- Failure and timeout tracking.
- Incident creation and recovery.
- Dashboard summaries, server status, history, and charts.
- Cloud deployment and documentation, subject to verified AWS access.

Out of scope initially

- A local agent for stdio-only MCP servers on a developer's laptop.
- Intercepting all tool calls made by arbitrary AI clients.
- A full MCP proxy or traffic-inspection product.
- Multi-tenant billing, enterprise SSO, or commercial SaaS features.
- Production-grade multi-region deployment, autoscaling, and high availability.

- Unconfirmed alert integrations or AWS services unavailable in the Learner Lab.

Important: Endpoint/protocol health checks do not reveal all tool calls made by an AI client. Actual tool-call analytics require client instrumentation or interception and are not part of the initial version.

5. User workflow

1. The user adds a remotely reachable MCP server with a name, endpoint, supported transport/protocol, and check interval.
2. The backend validates and stores the configuration.
3. A scheduler runs checks at the configured interval.
4. Each check records a timestamp, outcome, duration, and sanitized error information.
5. The dashboard shows current status, recent checks, and historical performance.
6. Repeated failures create or update an incident. Successful checks after an incident can mark it recovered according to the defined rule.

6. Proposed technology stack

Layer	Proposed technology / condition
Frontend	React + TypeScript + Tailwind CSS + Recharts
Backend/API	Node.js + Express + TypeScript
Database	PostgreSQL
Managed database	Amazon RDS only if available and permitted; otherwise a documented, approved alternative
Compute	Amazon EC2 if available; otherwise the simplest permitted AWS compute option
Object storage	Amazon S3 for exports/backups if useful and permitted
Logs/metrics	Amazon CloudWatch where available
Security/networking	IAM, VPC, and Security Groups as supported by the Learner Lab
Scheduler	Backend scheduler such as `node-cron` for the first version
Optional services	Lambda/EventBridge only if available and useful
Development	macOS, VS Code, Git, local development environment

Do not assume any AWS service is enabled. Verify permissions, regions, quotas, session limits, and cost implications before provisioning.

7. Logical cloud architecture

```
Browser
  |
  v
React + TypeScript Dashboard
  |
  v
Node.js + Express API / Monitoring Service
  |                       |
  |                       +---- Scheduled MCP checks ----> Remote MCP server
  |
  +----> PostgreSQL database
  |
  +----> CloudWatch logs/metrics (if available)

Optional:
  +----> S3 for exported reports/backups
```

Networking and security

- Expose only required web/API ports.
- Keep the database private where the architecture permits.
- Restrict database ingress to the application host/security group.
- Do not expose PostgreSQL publicly for convenience.
- Use least-privilege IAM permissions.
- Never commit credentials or place secrets in frontend bundles.
- Sanitize logs and stored errors.
- The monitored MCP server must be reachable from the monitoring runtime and support a transport/protocol implemented by the checker.
- A local stdio-only server is not remotely reachable by this architecture.

8. Functional requirements

ID	Requirement
FR-01	Create, view, edit, enable/disable, and delete monitored server configurations.
FR-02	Check endpoints using the selected MCP transport/protocol. Do not assume every MCP server has a generic HTTP health URL.
FR-03	Run checks automatically at a configurable interval. Prevent overlapping checks for the same server unless explicitly designed and tested.
FR-04	Store check timestamp, duration in milliseconds, result, and sanitized failure reason.
FR-05	Display status such as Unknown, Healthy, Degraded, or Down. Define exact thresholds and transitions.
FR-06	Display recent check history and response-time trends over a selected time range.
FR-07	Create incidents after a documented failure threshold; record start, reason, and recovery time.
FR-08	Optionally capture the list/count of tools exposed by a server when supported. This is metadata discovery, not usage analytics.
FR-09	Show server counts, status counts, recent incidents, and performance summaries.
FR-10	Validate input, protect management routes, and avoid logging credentials.
FR-11	Use structured, sanitized application logs.

FR-12 Optional lower-priority feature: CSV export of check history or incidents.

---

9. Non-functional requirements

- Separate frontend, API, monitoring/scheduler, and persistence concerns.
- A failed check must not crash the scheduler.
- Use least-privilege access, restricted inbound rules, input validation, and secret hygiene.
- Provide a responsive UI.
- Paginate or bound historical queries.
- Include setup instructions, environment-variable examples, database migrations, and deployment steps.
- Use AWS resources cost-consciously; stop/delete resources when not needed.
- Do not claim free-tier eligibility without verification.

10. Initial data model

Use migrations and foreign keys. Refine fields as needed without changing entity responsibilities.

`mcp_servers`

Candidate fields: `id`, `name`, `endpoint`, `transport_type`, `enabled`, `check_interval_seconds`, `created_at`, `updated_at`.

`monitoring_checks`

Candidate fields: `id`, `server_id`, `checked_at`, `duration_ms`, `outcome`, `status_code_if_applicable`, `error_code`, `sanitized_error`.

`incidents`

Candidate fields: `id`, `server_id`, `started_at`, `resolved_at`, `state`, `summary`, `triggering_check_id`.

Optional: `server_tool_snapshots`

Candidate fields: `id`, `server_id`, `captured_at`, `tool_count`, and normalized tool metadata.

Never store raw authorization headers, API keys, or sensitive request/response bodies in monitoring records. Use environment secrets or an approved secret mechanism for credentials.

11. API outline

Method Route Purpose

GET `/api/servers` List servers and latest status summary

POST `/api/servers` Create a validated server configuration

GET `/api/servers/:id` Get server details

PATCH `/api/servers/:id` Update configuration or enabled state

DELETE `/api/servers/:id` Delete a server according to documented history-retention behavior

GET `/api/servers/:id/checks` Paginated check history with time filters

GET `/api/servers/:id/incidents` Incident history

POST `/api/servers/:id/check-now` Request a protected manual check

GET `/api/dashboard/summary` Aggregate counts and recent incidents

---

12. UI pages

1. **Dashboard:** summary cards, server status table, recent incidents, response-time overview.
2. **Servers:** list/search with status, endpoint label, last check, and actions.
3. **Add/Edit Server:** validated name, endpoint, supported transport, and interval. Do not reveal saved secrets.
4. **Server Details:** current status, response-time chart, check history, incidents.
5. **Incidents:** active and historical incidents with start/recovery times and triggering details.
6. **Settings/About:** defaults, status/incident rules, and project information.

13. Monitoring semantics

- Define success for the selected MCP transport.
- Do not label a TCP ping or generic HTTP request as an MCP protocol check unless it verifies the protocol.
- Measure elapsed time in milliseconds.
- Set explicit connection and response timeouts.
- Sanitize errors before storing/displaying them.
- Make failure thresholds and recovery rules explicit and configurable.
- Define status transitions among Unknown, Healthy, Degraded (if implemented), and Down.
- Handle disabled, deleted, malformed, unsupported, and unreachable endpoints gracefully.
- Ensure one failed target does not stop checks for other targets.

14. AWS Learner Lab verification gate

Before provisioning or writing deployment-specific code, verify:

- Allowed AWS regions and services.
- Permitted EC2 instance types and session duration.
- Whether RDS is available.
- Whether IAM roles/policies and instance profiles can be used.
- CloudWatch log-group/metric permissions.
- S3 permissions and naming restrictions.
- VPC, subnet, routing, and security-group permissions.
- Budget and resource restrictions.

If a service is unavailable, document the substitution. Do not claim a managed AWS service was used if a local alternative was used.

15. Step-by-step implementation plan

Phase 0 --- Verify scope and access

1. Inspect the existing repository and report its current state.
2. Confirm the remote MCP test server, endpoint, transport, and authentication requirements.
3. Verify AWS Learner Lab permissions and constraints.
4. Choose the smallest deployment architecture that meets course requirements.
5. Define status transitions, failure threshold, recovery rule, and demo scenario.

Exit condition: Test endpoint and cloud assumptions are verified; architecture is documented.

Phase 1 --- Repository and local foundation

1. Create a documented frontend/backend structure or monorepo.
2. Set up TypeScript, linting/formatting, environment configuration, and README.

3. Create the Express health endpoint and React application shell.
4. Add `.env.example`; ensure real `.env` files are ignored by Git.
5. Verify frontend and backend run locally.

Exit condition: Frontend and backend start successfully with reproducible instructions.

Phase 2 --- Database and server registry

1. Set up PostgreSQL for local development.
2. Add migrations for servers, checks, and incidents.
3. Implement server CRUD APIs with validation.
4. Add pagination and clear API error handling.
5. Test validation, persistence, and failure cases.

Exit condition: A user can manage server configurations and data persists.

Phase 3 --- Monitoring engine

1. Implement a checker for the confirmed remote MCP transport.
2. Add timeouts, duration measurement, outcomes, and sanitized errors.
3. Implement scheduled checks and a manual check endpoint.
4. Persist results.
5. Prevent duplicate/overlapping checks where appropriate.
6. Implement incident creation and recovery rules.
7. Test success, failure, timeout, and recovery.

Exit condition: Scheduled monitoring records results and incidents end-to-end.

Phase 4 --- Dashboard and analytics

1. Build server list and status indicators.
2. Build server details, check history, and response-time chart.
3. Build incident views and dashboard summary.
4. Add loading, empty, and error states.
5. Keep historical queries bounded and responsive.

Exit condition: UI displays current/latest status and historical results from the backend.

Phase 5 --- AWS deployment

1. Provision only services confirmed available and explicitly approved.
2. Configure network access with least privilege.
3. Deploy frontend and backend using the simplest maintainable permitted setup.
4. Configure database connection and environment variables.
5. Configure CloudWatch and S3 only if permitted and justified.
6. Verify the deployed app can reach the monitored MCP endpoint.
7. Document deployment and teardown steps.

Exit condition: Deployed application works and cloud setup is documented accurately.

Phase 6 --- Testing and final submission

1. Test a healthy endpoint.
2. Simulate an outage or invalid endpoint.
3. Verify failure detection and incident creation.
4. Restore the endpoint and verify recovery.

5. Verify charts match stored data.
6. Review security, logs, and secret handling.
7. Prepare screenshots, architecture diagram, report, and demo script.
8. Document limitations and AWS substitutions.

Exit condition: End-to-end demo works, tests are recorded, and submission materials are ready.

16. Semester timeline

The exact submission date and remaining weeks are not specified. Use this relative six-week plan and compress or extend it based on time remaining.

Period Target

Week 1 Requirements, AWS access, test endpoint, project setup Week 2 Database, server registry, backend APIs Week 3 Monitoring engine, scheduler, incidents Week 4 Dashboard, charts, history Week 5 AWS deployment and cloud configuration Week 6 Testing, fixes, report, screenshots, demo

Do not leave deployment and testing until the final days.

17. Acceptance criteria

- A user can add and manage at least one remote MCP server configuration.
- Monitoring performs scheduled checks without a manual browser action.
- Each check stores timestamp, duration, outcome, and sanitized error information when applicable.
- Dashboard displays latest status and historical response-time data.
- A simulated outage produces a recorded failure and incident according to the documented threshold.
- Restoring the endpoint updates status and resolves the incident according to the recovery rule.
- The application deploys in the available AWS environment, or restrictions are clearly documented.
- No real credentials are committed or exposed in logs.
- README allows another developer to run the project locally and understand deployment.

18. Testing checklist

- Valid and invalid endpoint input.
- Unreachable endpoint, connection timeout, response timeout, malformed response.
- Scheduler continues after a check failure.
- Duplicate/overlapping check prevention.
- Incident creation and recovery transitions.
- Clean database migration.
- API pagination and time filtering.
- Dashboard loading, empty, and error states.
- Management endpoint protection appropriate to the chosen authentication design.
- Secret redaction in logs and stored errors.
- AWS security group and database network access review.

19. Instructions for Antigravity

Treat this document as the project source of truth.

- Start by inspecting the existing repository; report the stack, files, and implementation state.
- Work in small, reviewable phases. Do not generate the entire project at once.

- Do not invent AWS permissions, endpoint behavior, credentials, or service availability.
- Ask for clarification when a decision blocks implementation.
- Do not implement the MCP checker until the remote test server and its transport/protocol are confirmed.
- Keep the initial scope limited to remote monitoring, dashboard analytics, and cloud deployment.
- Do not implement local stdio monitoring or AI-client tool-call interception in the initial version.
- Do not add Kubernetes, Kafka, microservices, complex alerting, or unrelated features without a concrete requirement.
- Use migrations, validation, typed interfaces, tests, and useful error handling.
- Never put secrets in source code, example files, frontend bundles, screenshots, or logs.
- Do not provision AWS resources or incur costs without explicit user approval.
- If a service is unavailable, explain the limitation and propose the smallest compliant alternative.
- At the end of each phase, report changed files, commands to run, tests actually performed, known issues, and the next phase.
- Do not claim tests passed unless they were run.

20. Initial prompt for Antigravity

Copy and paste this after uploading this Markdown file:

> We are building MCPulse --- MCP Server Monitoring & Performance Analytics Platform. Read `MCPulse_Project_Specification.md` completely and treat it as the source of truth.
>
> The project is primarily about Cloud Computing, using AWS where permitted by our AWS Learner Lab. The initial approach is cloud-based monitoring of remotely accessible MCP servers. Do not build a local stdio monitoring agent or AI-client tool-call interceptor.
>
> **For now, inspect and plan only. Do not modify files yet.**
>
> Report: 1. The existing repository structure, stack, dependencies, and implementation state. 2. Decisions that block implementation, especially the remote MCP test server/transport and AWS service permissions. 3. The smallest sensible Phase 0 and Phase 1 plan. 4. The exact files you propose to create or modify and the commands needed to run them.
>
> Constraints: - Do not invent AWS permissions, credentials, endpoints, or protocol behavior. - Do not provision AWS resources or incur costs without my explicit approval. - Use the proposed stack where appropriate: React, TypeScript, Tailwind CSS, Recharts, Node.js, Express, and PostgreSQL. - Keep AWS services conditional on verified Learner Lab access. - Use environment variables, validate inputs, protect secrets, and include tests and a README. - Prefer a small, working end-to-end implementation over unnecessary complexity.
>
> Wait for my approval before starting implementation.

21. Definition of done

MCPulse is complete when the accepted scope runs locally, the monitoring workflow is tested end-to-end, cloud deployment uses only confirmed and available services, the demonstration shows failure and recovery, and the repository contains reproducible setup/deployment instructions and a clear account of limitations.